

REAL TIME CHAT APP MIDTERM REPORT

Using HTML, CSS, JavaScript, React and Firebase

Siddhant Patankar

2583908

Duration:

4 Weeks

Technology Stack:

HTML, CSS, JavaScript, React.js, Firebase Authentication

PROJECT OVERVIEW

The objective of this four-week training program was to learn modern web development technologies and progressively build a chat application. The project started with designing a static user interface using HTML and CSS, followed by adding dynamic functionality using JavaScript. The application was then converted into a React-based component architecture and finally integrated with Firebase Authentication to enable secure Google Sign-In functionality.

The training emphasized both theoretical understanding and practical implementation. Throughout the four weeks, several industry-standard concepts such as Flexbox layouts, DOM manipulation, React components, Context API, asynchronous programming, and cloud-based authentication were explored and applied.

WEEK 1

During the first week, the focus was on understanding the basic building blocks of web development. HTML was used to create the structure of web pages, while CSS was used to improve presentation and visual appearance.

The following topics were studied:

- HTML document structure
- Headings, paragraphs, lists and forms
- Div containers and semantic tags
- CSS selectors and properties
- Box Model (margin, border, padding, content)
- Flexbox layout system
- Responsive web design
- CSS Grid fundamentals
- Git and GitHub basics

Understanding Flexbox

One of the most important concepts learned during this week was Flexbox. Flexbox simplifies page layouts by allowing elements to be arranged horizontally or vertically while automatically adjusting spacing and alignment.

In the chat application, Flexbox was used to divide the page into two sections:

- Left Sidebar
- Right Chat Area

This allowed the layout to remain clean and responsive across different screen sizes.

Implementation of the Chat Interface

The task involved creating a static chat interface consisting of:

1. Sidebar Section
 - Application logo
 - Search bar
 - Contact list
2. Chat Section
 - Chat header
 - Message display area
 - Message input box

The layout was designed using Flexbox to ensure proper alignment and spacing.

-Responsive Design

Media queries were used to improve the mobile experience. When the screen width becomes smaller, the sidebar automatically hides and the chat section occupies the entire screen width.

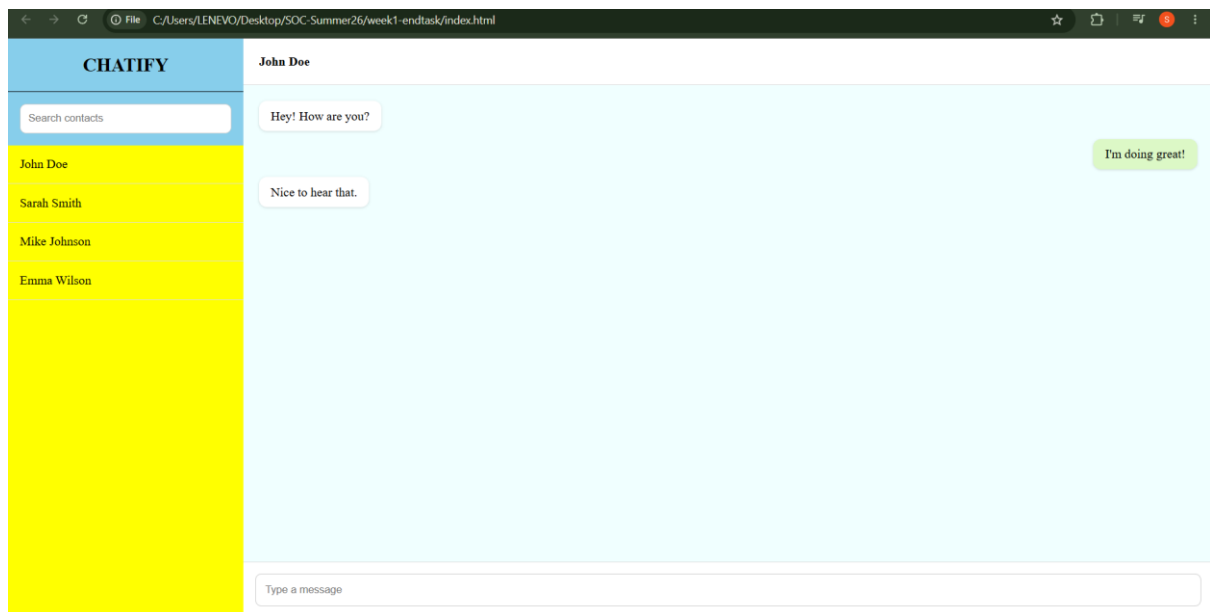
-Challenges Faced

Initially, understanding height inheritance and Flexbox behavior was difficult. The sidebar and contact list were not expanding to the full page height because parent containers were not assigned proper heights. This issue was resolved by learning how height propagation works in CSS and by using viewport height units effectively.

-Learnings from Week 1

- Developed a strong understanding of HTML page structure.
- Learned to design layouts using Flexbox.
- Understood responsive design principles.
- Learned how CSS styling impacts user experience.
- Gained familiarity with Git and version control concepts.

WEEK 1 OUTPUT:



WEEK 2

The second week focused on adding interactivity to web pages using JavaScript. Unlike HTML and CSS, JavaScript enables dynamic behavior and user interaction.

The following concepts were studied:

- Variables and data types
- Functions
- Conditional statements
- Loops
- DOM manipulation
- Event listeners
- Asynchronous JavaScript
- Callbacks
- Promises
- Async/Await

-DOM Manipulation

A major focus was learning how JavaScript interacts with HTML elements through the Document Object Model (DOM).

Methods such as:

- `document.querySelector()`
- `document.createElement()`
- `appendChild()`

were used extensively during the task.

-Implementation of Chat Functionality

The static chat application from Week 1 was enhanced with dynamic functionality.

Features implemented:

- Reading user input from the message box
- Creating new message elements dynamically
- Appending messages to the chat area
- Adding timestamps to each message
- Clearing the input field after sending
- Automatically scrolling to the latest message

-Asynchronous JavaScript

Concepts such as callbacks, promises, and `async/await` were explored to understand how JavaScript handles operations that take time to complete.

Although the chat application itself did not require complex asynchronous operations, this knowledge later became useful during Firebase integration.

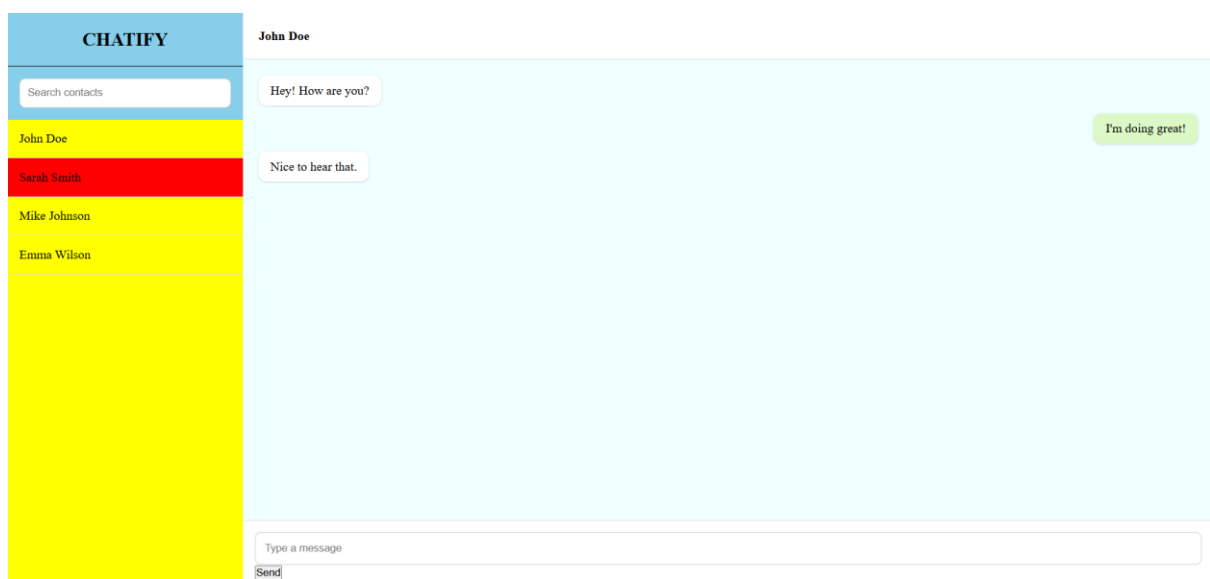
-Challenges Faced

One challenge encountered was debugging situations where newly created message elements were not appearing on the page. Through browser developer tools and console logging, it became easier to identify issues related to incorrect element selection and DOM insertion.

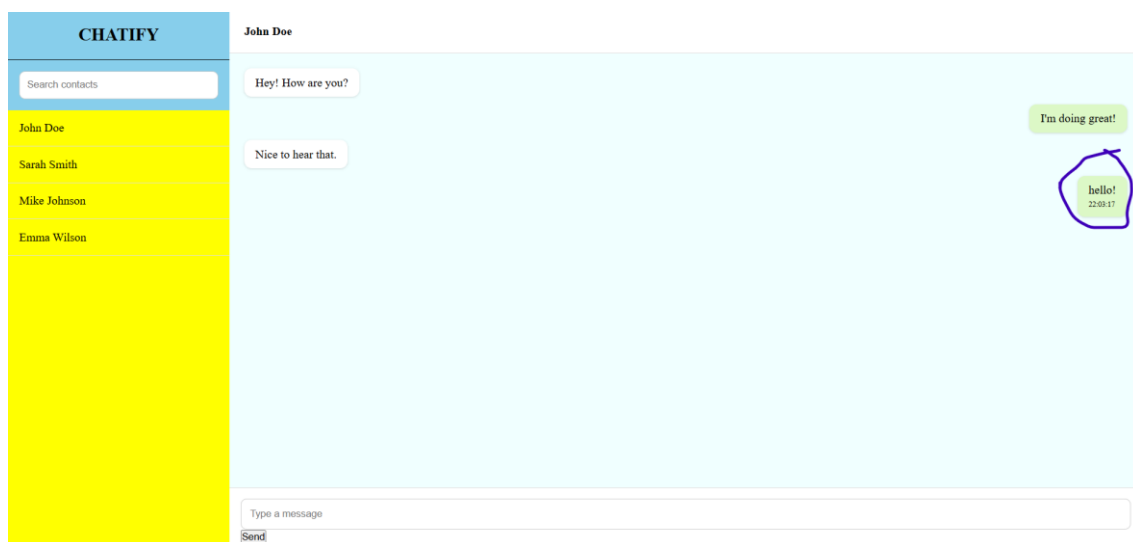
-Learnings from Week 2

- Learned how JavaScript interacts with webpage elements.
- Understood event-driven programming.
- Gained practical experience with DOM manipulation.
- Learned debugging techniques using browser developer tools.
- Developed a foundation in asynchronous programming.

WEEK 2 OUTPUT:



After I type in hello! and hit send :



WEEK 3

The third week introduced React, a popular JavaScript library used for building modern user interfaces.

Topics covered included:

- React fundamentals
- JSX syntax
- Functional components
- Props
- Component composition
- React Hooks overview
- Project structure and organization

Component-Based Architecture

Unlike traditional HTML pages, React applications are built using reusable components.

The chat application was divided into multiple components:

- Sidebar.jsx
- ChatWindow.jsx
- MessageBubble.jsx
- MessageInput.jsx

This improved code organization and maintainability.

-Understanding JSX

JSX allows HTML-like syntax to be written directly inside JavaScript files.

Example:

Instead of manually creating DOM elements, React components describe the UI declaratively using JSX.

-Props

Props were introduced as a mechanism for passing information between components.

For example, MessageBubble received:

- Message text
- Message direction (sent or received)

This enabled reusable message rendering.

-Application Assembly

All individual components were imported and assembled inside App.jsx, creating a complete chat application using React's component-based structure.

-Challenges Faced

Initially, understanding how components communicate and how JSX differs from traditional HTML required additional practice. However, building the project component by component made the overall architecture easier to understand.

-Learnings from Week 3

- Learned React fundamentals and component structure.
- Understood JSX syntax.
- Learned how props enable reusable UI components.
- Improved project organization through modular design.
- Gained exposure to modern frontend development workflows.

WEEK 3 OUTPUT:



WEEK 4:

The final week focused on integrating Firebase into the React application to add authentication functionality.

Topics covered:

- Firebase project setup
- Firebase SDK installation
- Google Authentication
- Context API
- Protected Routes
- User state management

-Firebase Configuration

A Firebase project was created through the Firebase Console. The application was then registered as a web application, and the Firebase configuration object was integrated into the React project.

-Authentication Setup

Google Sign-In was enabled through Firebase Authentication.

This allowed users to securely sign in using their Google accounts without manually managing passwords.

-React Context API

To manage authentication state globally, an AuthContext component was created.

The Context API allowed user information to be shared across multiple components without repeatedly passing props through the component hierarchy.

-Protected Routes

A ProtectedRoute component was implemented to ensure that only authenticated users could access the chat application.

If a user was not logged in, they were automatically redirected to the login page.

-User Profile Integration

After successful authentication, the user's:

- Name
- Profile Picture

were displayed in the sidebar header, creating a personalized user experience.

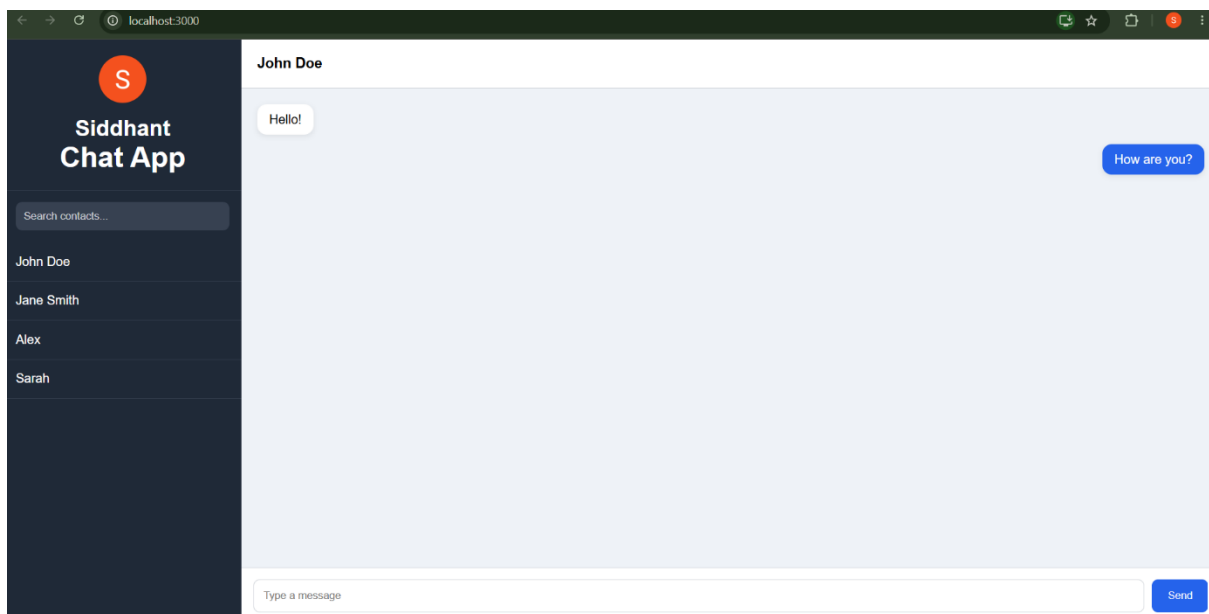
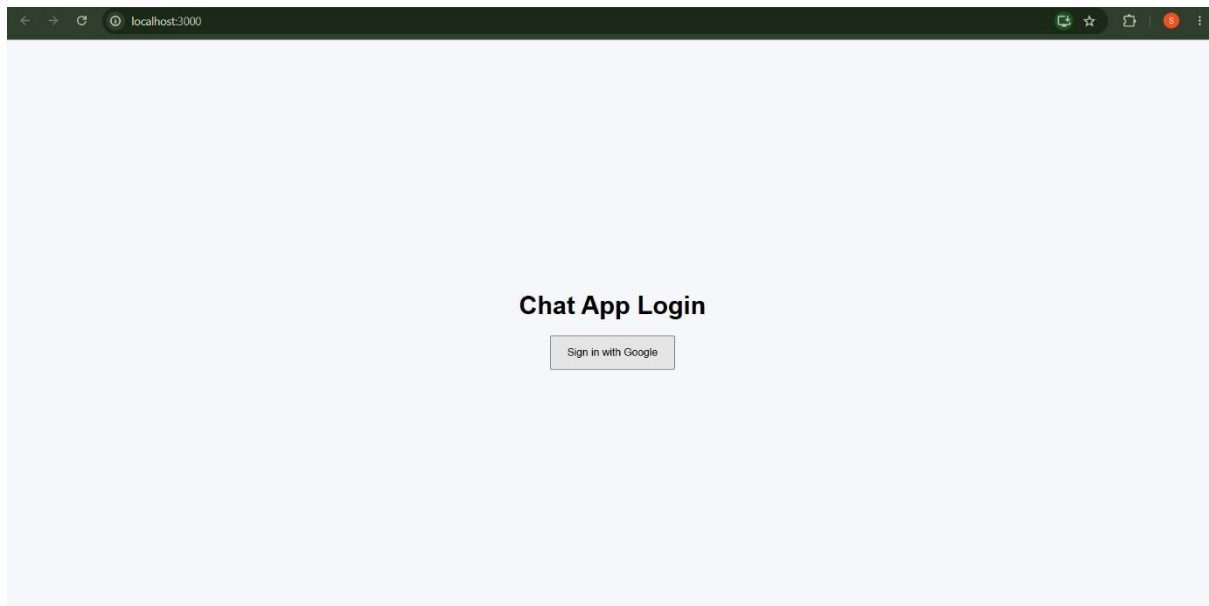
-Challenges Faced

The most significant challenge was configuring Firebase Authentication correctly. Initial issues related to authentication provider settings and Firebase configuration required debugging and verification through the Firebase Console.

-Learnings from Week 4

- Learned cloud-based authentication concepts.
- Understood Firebase project configuration.
- Implemented Google Sign-In functionality.
- Learned React Context API.
- Implemented protected routes and user state management.
- Gained practical experience integrating third-party services.

WEEK 4 OUTPUT: (before and after signing in with Google)



Key skills acquired during these 4 weeks include:

- HTML page structuring
- CSS styling and responsive design
- JavaScript DOM manipulation
- Event-driven programming
- React component development
- Context API usage
- Firebase Authentication integration
- Debugging and problem-solving

Conclusion:

The chat application project till now successfully combined HTML, CSS, JavaScript, React, and Firebase into a single application. Throughout the four weeks, theoretical concepts were reinforced through practical implementation, resulting in a deeper understanding of modern web development practices.

The experience improved both technical skills and problem-solving abilities while providing exposure to industry-relevant tools and frameworks. The knowledge gained during this training serves as a strong foundation for developing more advanced web applications in the future.